

Analysis Report: An Efficient, Analyzed Ride-Sharing Simulator

Raymond E. Ogg, II

Introduction

This project models an on-demand ride-sharing dispatch service as a discrete-event simulation. The objective was to measure how efficiently a fixed fleet of cars serves a dynamic, unpredictable stream of ride requests across a road network, and to observe the behavior of the data structures and algorithms that make that matching possible.

The simulation was built to answer two central questions. First, is the service efficient from the operator's perspective; are drivers kept productive, and does the system complete the demand it receives? Second, is the rider experience acceptable; how long do riders wait, and how often does a request go unserved? A recurring theme in the analysis is the tension between these two goals.

Methodology

The simulation is built on an event-driven engine that advances time through a min-heap priority queue of events. Each event is a four-field tuple: timestamp, a strictly increasing sequence number, an event type, and its payload (where the sequence number acts as a tie-breaker so that events sharing a timestamp are processed deterministically and the heap never attempts to compare Car or Rider objects). Three event types drive the system: rider request, pickup arrival, and drop-off arrival.

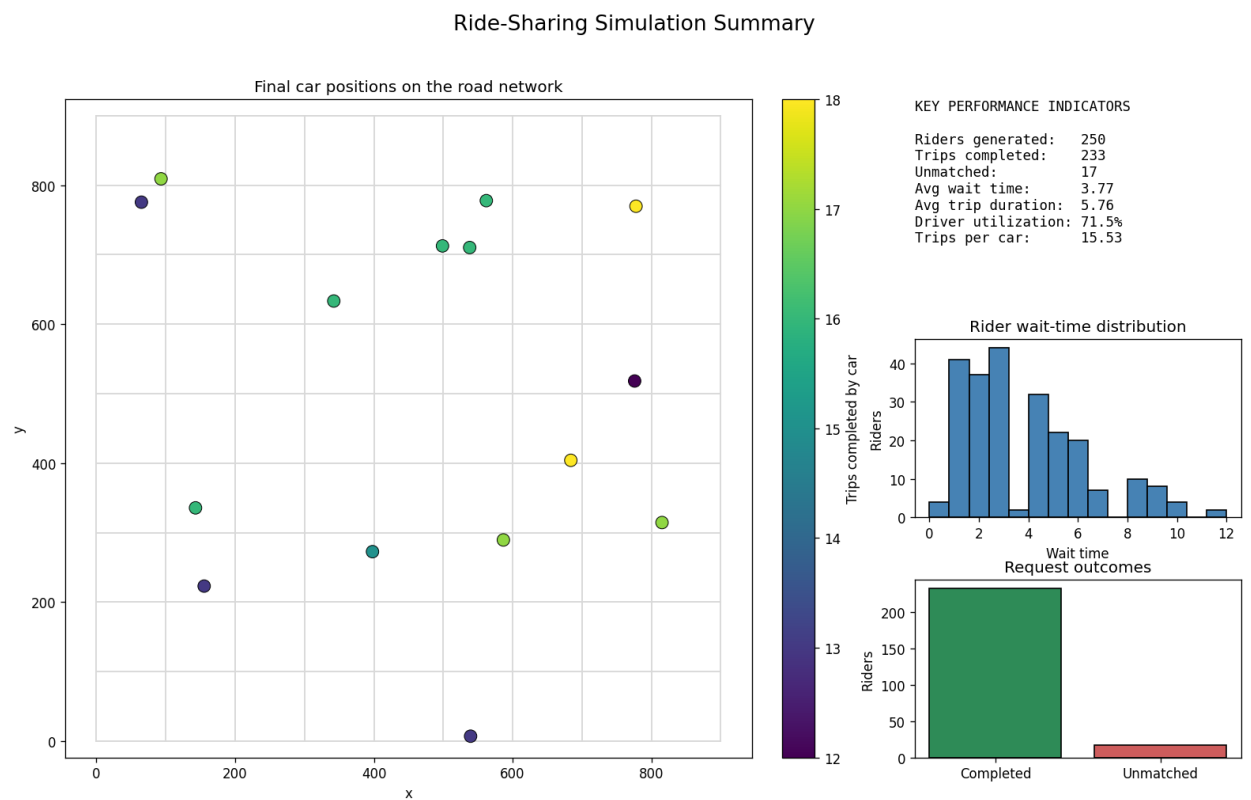
The road network is stored as a Graph holding both a weighted adjacency list and the physical coordinates of every node. Matching a rider to a car uses a two-stage pipeline. A Quadtree spatial index returns up to k geographically nearest available cars ($k = 5$ by default); Dijkstra's algorithm is then run from each of those candidates to the rider, and the reachable candidate with the lowest actual road-network travel time is selected. Availability is tracked by three structures: a car-lookup dictionary, a point-lookup dictionary, and the Quadtree. These are kept synchronized through two centralized methods so a dispatched car is cleanly absent from all three until drop-off.

These choices were deliberate relative to the alternatives considered in the course. A min-heap gives $O(\log n)$ insertion and extraction for the event queue, where an unsorted list would require an $O(n)$ scan for the next event and a sorted list an $O(n)$ insertion. Dijkstra's algorithm is the appropriate shortest-path method for a graph with non-negative weights: a breadth-first search would be correct only on unweighted or uniform-cost edges, while Bellman-Ford's $O(V \cdot E)$ cost is justified only when negative weights are present, which they are not here. For spatial matching, the Quadtree prunes whole regions of the map during a nearest-neighbor query, avoiding the $O(n)$ linear scan of the entire fleet that a naive search would require on every request.

The results below come from a single reproducible run on the production map (city_map.csv, a 10×10 grid of 100 intersections connected by 180 bidirectional unit-weight roads). The run used a fleet of 15 cars, a generation cap of 250 riders, a maximum simulation time of 1000 units, an exponential mean inter-arrival interval of 0.8 time units, a candidate count of $k = 5$, and random seed 42 for reproducibility. These variables can be adjusted at-will during startup, including surge pricing which would be reflected in the KPI section.

Results

The integrated visualization below shows the final position of every car on the road network (a heat map is used to show how many trips a given car made), the summary KPI panel, and two charts: the distribution of rider wait times and the count of completed versus unmatched requests.



The key performance indicators for this run were as follows:

Key Performance Indicator	Value
Total completed trips	233
Average rider wait time	3.77 time units
Average rider trip duration	5.76 time units
Overall driver utilization	71.5%
Total riders generated	250

Unmatched / unsuccessful riders	17 (6.8%)
Trips completed per car	15.53

In summary, the simulation run with 15 cars over a 1000-unit horizon generated 250 riders, of which 233 completed their trips and 17 went unmatched, producing an average rider wait time of 3.77 time units, an average trip duration of 5.76 time units, and an overall driver utilization of 71.5%.

Discussion

A driver utilization of 71.5% indicates an efficient system from the operator's perspective: the fleet spends most of the simulation actively serving riders rather than sitting idle. Each car completed roughly 15 trips, and the great majority of demand (233 of 250 requests) was fulfilled. On the business side, this is a healthy result.

The rider experience is a little more of gray-area. The average wait of 3.77 time units is certainly short relative to the average trip duration of 5.76, but the wait-time distribution is right-skewed: while most riders were matched quickly, a tail of riders waited far longer, and 17 requests (6.8%) received no car at all. This is the central trade-off the data reveals. The same 15-car fleet that produces high utilization is precisely what stretches thin during demand peaks; the unmatched requests and the long wait tail are the cost of running the fleet near its capacity. High driver utilization and a uniformly good rider experience are in direct opposition, pushing utilization higher by shrinking the fleet would worsen both the wait tail and the unmatched rate, while improving the rider experience by adding cars would leave those cars idle more often.

Judged against a service standard, the mean wait understates the problem: the riders who matter most for a quality assessment are those in the tail and the 6.8% who were never served. By that measure the service is efficient but stretched, operating close to the point where demand exceeds what the fleet can absorb without degradation.

Several simplifications limit how directly these results map to a real system. Edge weights are constant, so there is no traffic, congestion, or time-of-day variation in travel time. Each trip has a single origin and destination; there is no ride pooling or multi-stop routing. Matching is instantaneous, demand is drawn uniformly at random across the map rather than concentrated in realistic hotspots, and the model omits fuel stops, driver shifts and breaks, and rider cancellations. Cars also do not reposition proactively toward anticipated demand. Each of these factors would reduce effective supply or lengthen trips, which would raise wait times and the unmatched rate above what this idealized model reports.

Increasing the number of cars should lower utilization while improving the rider experience, and reducing it should do the opposite. Exploratory runs on the same map and seed support this direction. Raising the fleet to 20 cars dropped utilization to roughly 48% while eliminating unmatched requests, and 25 cars fell to about 36% with no unmatched riders. Conversely, cutting

the fleet further would drive utilization toward saturation and drastically increase the unmatched count. The relationship shows diminishing returns one would expect in the real-world: if demand is met, increasing the “supply” side (e.g. cars) causes them to simply sit idle.

As an extension, a zone-based surge-pricing model was implemented: the map is partitioned into zones, and a fare multiplier is applied to each trip based on its origin zone's ratio of requests to available drivers. Over this run the average multiplier was $1.77\times$, the maximum reached the $3.00\times$ cap in the busiest zones, and the average fare was 8.85. Notably, surge pricing did not change wait time or utilization at all. Because the fleet is fixed and riders in this model do not decline a ride based on price, the fare is computed and charged but never feeds back into which car is dispatched and to where. The surge model is therefore descriptive, it correctly prices demand hotspots, but does not model behaviors.

Conclusion

A 15-car fleet on this network achieved high driver utilization (71.5%) but at the cost of a right-skewed wait-time distribution and a 6.8% unmatched rate, illustrating the fundamental trade-off between operator efficiency and rider experience. The most valuable next improvements would be a waiting queue that re-matches temporarily unmatched riders as cars free up (which would lower the un-matched rate and reflect reality a bit better) and traffic-weighted, time-varying edge costs to make travel times realistic. Extending the surge model with a behavioral response, where either driver supply or rider demand reacts to price, would also let pricing influence the service metrics it currently only observes. Given an additional week, I believe those would be the two things I would absolutely improve on.